

HELIXCLUSTER PHASE 2 — CONSOLE COMPUTE NODES

PlayStation 3/4/4 Pro/5/5 Pro Integration Architecture

Version 1.0 | 2026-05-30

1. EXECUTIVE SUMMARY

HelixCluster Phase 2 extends the distributed computing cluster to include jailbroken PlayStation consoles (PS4, PS4 Pro, PS5, PS5 Pro) as fully integrated worker nodes. **PS3 is excluded** — its Cell BE architecture is obsolete and programming complexity is prohibitive.

Key Value Proposition

Metric	PS4 Pro Node	PS5 Node	Desktop PC Equivalent
Cost (used)	\$150-250	\$400-500	\$600-1000
GPU TFLOPS	4.2	10.3	4-10
CPU	8c Jaguar @ 2.1GHz	8c/16t Zen2 @ 3.5GHz	Varies
RAM	8GB GDDR5	16GB GDDR6	16GB DDR4
GPU \$/TFLOP	~\$59	~\$49	~\$100+
Power	~160W	~200W	~300W+

Unique Capabilities Consoles Bring

- GPU Compute at 1/2 the PC cost** — Discarded gaming hardware repurposed for compute
- PS5 Custom I/O Decompressor** — 8-9 GB/s hardware decompression (no PC equivalent)
- GDDR5/GDDR6 unified memory** — Higher bandwidth than DDR4 for GPU workloads
- Disposable node model** — At \$80-250, failed nodes are replaced, not repaired
- Community contribution** — Users can donate idle console time (Folding@home model)

2. SUPPORTED CONSOLE HARDWARE MATRIX

2.1 Console Tier Classification

Console Tier Classification			
Tier	Hardware	Suitability	Cluster Role
Tier-1 (Premium)	PS5 / PS5 Pro	EXCELLENT	GPU Compute Ace
	Zen2 8c/16t		AI Inference Primary
	RDNA2 10-33 TF		High-Perf Batch Jobs
	16GB GDDR6		PS5 I/O Decompression
Tier-2 (Standard)	PS4 Pro	GOOD	GPU Compute Worker
	Jaguar 8c		Batch Job Worker

	GCN 4.2 TF		Build Farm Node
	8GB GDDR5		Video Transcode
TIER-3 (Basic)	PS4 Base Jaguar 8c	ADEQUATE	Lightweight Worker Cache/Storage Node
	GCN 1.84 TF		Network Relay
	8GB GDDR5		Fallback Compute
EXCLUDED	PS3 / PS3 Slim	POOR	Not Supported
	Cell BE 7 SPEs		
	256MB XDR RAM		

2.2 Detailed Hardware Specifications

PS4 Base (CUH-10xx/11xx series)

```
console:
  model: "PS4 Fat"
  tier: 3
  soc: "AMD Liverpool"

cpu:
  architecture: "AMD Jaguar x86-64"
  cores: 8
  threads: 8
  clock: "1.6 GHz base (up to 1.75 boost)"
  features: ["AES-NI", "AVX", "SSE4.1", "SSE4.2"]
  # NOTE: NO AVX2

gpu:
  architecture: "AMD GCN 2.0 (Liverpool)"
  compute_units: 18
  tflops_fp32: 1.84
  memory: "Shared with system"

memory:
  type: "GDDR5"
  total: "8 GB"
  available_linux: "~6.85 GB"
  bandwidth: "176 GB/s"

storage:
  internal: "500GB-1TB SATA HDD"
  upgradeable: "2.5" SATA SSD"

network:
  ethernet: "Gigabit (Realtek RTL8153)"
  wifi: "802.11 b/g/n (2.4GHz only)"
  bluetooth: "2.1+EDR"
  usb: "USB 3.0 x2, USB 2.0 x1"
```

```
power:
  tdp: "~120W under load"
  idle: "~60W"

cost:
  used_market: "$80-150"

linux_support:
  status: "MATURE"
  kernel: "6.15.4 (latest)"
  gpu_accel: "Full (AMDGPU + Mesa)"
  docker: "Yes"
  kvm: "No (PS4 base)"

limitations:
  - "No AVX2 (limits some optimized code)"
  - "Weak single-thread (~1400 Geekbench 4 SC)"
  - "Thermal throttling common (needs maintenance)"
  - "SATA only (no NVMe)"
```

PS4 Pro (CUH-70xx series)

```
console:
  model: "PS4 Pro"
  tier: 2
  soc: "AMD Neo"

cpu:
  architecture: "AMD Jaguar x86-64"
  cores: 8
  threads: 8
  clock: "2.13 GHz (OC to 2.6 GHz stable)"
  features: ["AES-NI", "AVX", "SSE4.1", "SSE4.2"]
  # NOTE: NO AVX2

gpu:
  architecture: "AMD GCN 4.0 Polaris"
  compute_units: 36
  tflpos_fp32: 4.20
  memory: "Shared with system"

memory:
  type: "GDDR5"
  total: "8 GB GDDR5 + 1 GB DDR3"
  available_linux: "~6.85 GB GDDR5"
  bandwidth: "218 GB/s"

storage:
  internal: "1TB SATA HDD"
  upgradeable: "2.5" SATA SSD"
```

```
network:
  ethernet: "Gigabit (Realtek RTL8153)"
  wifi: "802.11 a/b/g/n/ac"
  bluetooth: "4.0"
  usb: "USB 3.1 Gen1 x3"

power:
  tdp: "~160W under load"
  idle: "~70W"

cost:
  used_market: "$150-250"

linux_support:
  status: "MATURE"
  kernel: "6.15.4 (latest)"
  gpu_accel: "Full (AMDGPU + Mesa, some accel issues)"
  docker: "Yes"
  kvm: "YES (confirmed working)"

limitations:
  - "No AVX2"
  - "GPU acceleration has minor issues under Linux"
  - "SATA only (no NVMe)"

advantages:
  - "Best cost/performance ratio ($59/TFLOP)"
  - "KVM enables nested virtualization"
  - "WiFi AC for wireless cluster nodes"
  - "Overclockable CPU to 2.6 GHz"
```

PS5 Base (CFI-10xx/11xx series)

```
console:
  model: "PS5"
  tier: 1
  soc: "AMD Oberon"

cpu:
  architecture: "AMD Zen 2 x86-64"
  cores: 8
  threads: 16
  clock: "3.5 GHz (variable frequency)"
  features: ["AES-NI", "AVX2", "AVX-512? No", "SSE4.2"]
  note: "35% smaller FPU than desktop Zen 2"

gpu:
  architecture: "AMD RDNA 2"
  compute_units: 36
  tflops_fp32: 10.28
```

```
ray_tracing: "Yes (hardware)"
memory: "Shared with system"

memory:
  type: "GDDR6"
  total: "16 GB"
  available_linux: "~12-13 GB"
  bandwidth: "448 GB/s"

storage:
  internal: "825GB custom NVMe SSD"
  expansion: "M.2 NVMe slot (PCIe 4.0 x4)"
  speed_raw: "5.5 GB/s"
  speed_compressed: "8-9 GB/s (Kraken hardware)"

network:
  ethernet: "Gigabit"
  wifi: "WiFi 6 (802.11ax)"
  bluetooth: "5.1"
  usb: "USB 3.1 Gen2 x2, USB 2.0 x2, USB-C"

# CUSTOM I/O COMPLEX (Unique Advantage)
custom_io:
  decompressor: "Kraken/Zlib/Oodle hardware"
  throughput: "8-9 GB/s compressed → raw"
  equivalent_cpu: "~9 Zen 2 cores"
  accessible_linux: "NO (Orbis OS only)"
  accessible_orbis: "YES"

power:
  tdp: "~200W under load"
  idle: "~50W"

cost:
  used_market: "$400-500"

linux_support:
  status: "NEW (April 2026)"
  kernel: "6.8+ (Ubuntu 24.04)"
  gpu_accel: "Full (AMDGPU + Mesa)"
  docker: "Yes"
  kvm: "Expected (Zen 2 has AMD-V)"

limitations:
  - "Jailbreak limited to firmwares 3.xx-4.xx (best) or 7.61 (max)"
  - "Custom I/O decompressor inaccessible from Linux"
  - "Smaller FPU than desktop Zen 2"
  - "Variable frequency may affect benchmarking"

advantages:
  - "Desktop-class CPU (Zen 2 8c/16t)"
```

- "Excellent GPU compute (RDNA2)"
- "16GB GDDR6 with 448 GB/s bandwidth"
- "WiFi 6 for high-speed wireless nodes"
- "Custom NVMe SSD + M.2 expansion"
- "Hardware decompression via Orbis OS agent"

PS5 Pro

```
console:
  model: "PS5 Pro"
  tier: 1
  soc: "AMD Oberon Plus"

cpu:
  architecture: "AMD Zen 2 x86-64"
  cores: 8
  threads: 16
  clock: "Up to 3.85 GHz"

gpu:
  architecture: "AMD RDNA 2 Extended"
  compute_units: 60
  tflops_fp32: "33.5 (estimated)"
  ray_tracing: "Yes (enhanced)"

memory:
  type: "GDDR6"
  total: "16 GB GDDR6 + 2 GB DDR5"
  bandwidth: "576 GB/s"

# AI PROCESSING UNIT (Unique)
ai_unit:
  type: "PSSR (PlayStation Spectral Super Resolution)"
  performance: "300 TOPS INT8"
  accessible: "Unknown from Linux"

storage:
  internal: "2TB custom NVMe SSD"
  expansion: "M.2 NVMe slot"

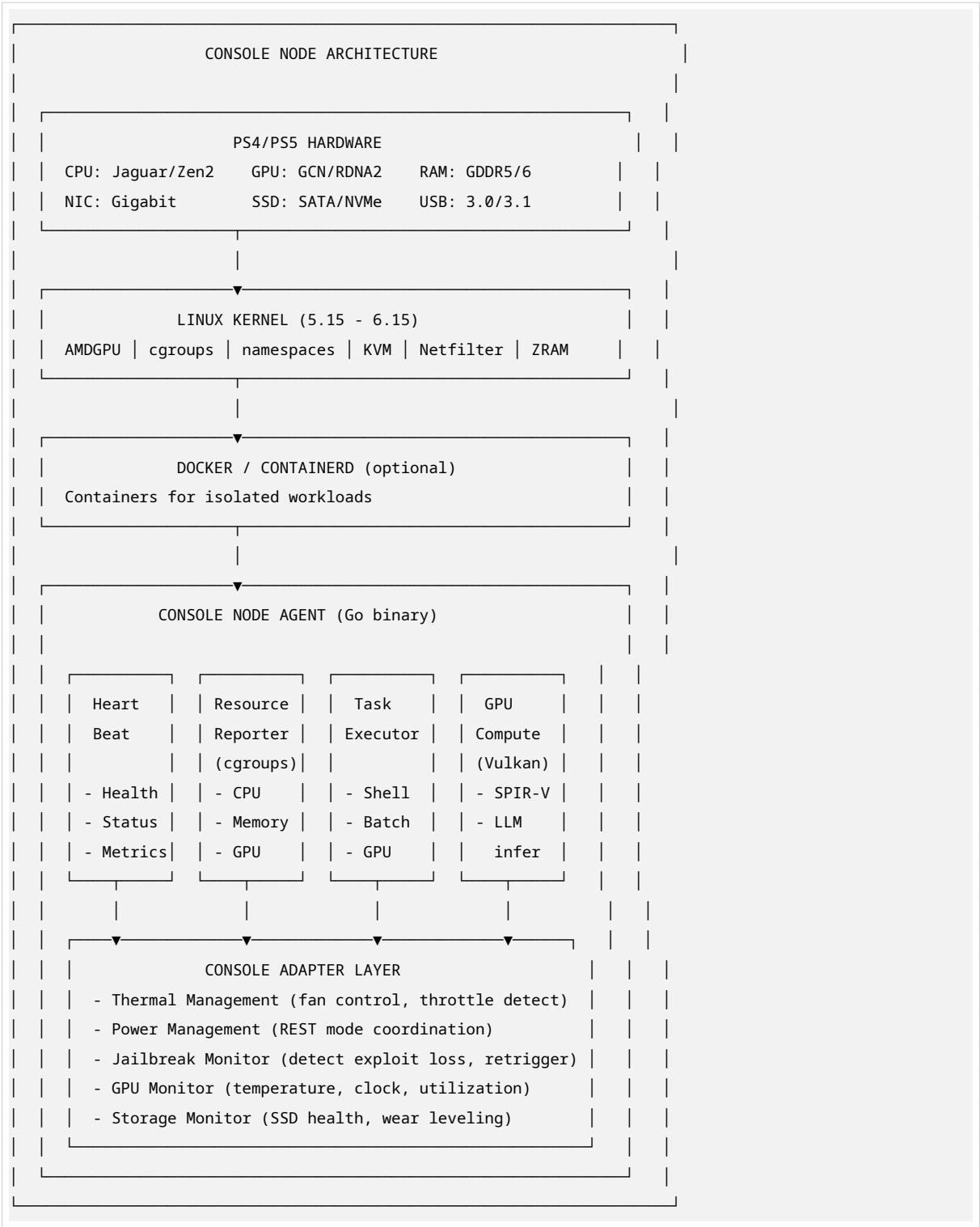
cost:
  used_market: "$550-700 (rare)"

linux_support:
  status: "VERY NEW"
  note: "Userland exploit up to 10.40, kernel exploit limited"
```

3. CONSOLE NODE AGENT ARCHITECTURE

3.1 Node Agent Deployment Model

Each console runs a **minimal Linux distribution** (psxitech, Fedora, or Ubuntu) with our Console Node Agent as a systemd service. The agent provides the same interface as PC node agents but adapts to console-specific constraints.



3.2 Console Adapter Layer

The **Console Adapter Layer** is unique to console nodes and handles console-specific concerns:

```
// Console Adapter Interface
package console

type ConsoleAdapter interface {
    // Thermal Management
    GetTemperature() (*ThermalState, error)
    SetFanSpeed(percent int) error
    GetThermalThrottleStatus() (bool, error)

    // Power Management
    GetPowerConsumption() (watts float64, error)
    EnterRestMode() error // Preserve jailbreak during idle
    WakeFromRestMode() error

    // Jailbreak Management
    IsJailbroken() (bool, error)
    GetJailbreakVersion() (string, error)
    TriggerExploit() error // Retrigger if lost
    AutoExploitEnabled() (bool, error)

    // GPU Monitoring (console-specific)
    GetGPUClock() (mhz int, error)
    GetGPUUtilization() (percent int, error)
    GetVRAMUsage() (used, total int64, error)
    GetGPUTemperature() (celsius int, error)

    // Storage
    GetSSDHealth() (*SSDHealth, error)
    GetStorageType() StorageType // SATA SSD / NVMe
}

type ThermalState struct {
    CPUCelsius      int
    GPUCelsius      int
    FanSpeedPercent int
    Throttling      bool
}

type SSDHealth struct {
    WearLevel      int // 0-100%
    BadBlocks      int
    PowerOnHours   int
    RemainingLife  int // Estimated days
}
```

3.3 Console-Specific Resource Reporting

```
// ConsoleNodeResources extends NodeResources with console-specific fields
```



```
type ConsoleNodeResources struct {
    BaseResources NodeResources // Standard CPU/Mem/GPU

    ConsoleSpecific struct {
        Model          ConsoleModel // PS4_FAT, PS4_PRO, PS5, PS5_PRO
        Firmware        string      // e.g., "9.00", "4.51"
        JailbreakType   string      // "GoldHen", "etaHEN", "UMTX2"
        JailbreakVersion string

        Thermal struct {
            CPUCurrentC    int
            GPUCurrentC    int
            FanSpeedPct    int
            Throttling      bool
        }

        Power struct {
            CurrentWatts    float64
            AverageWatts    float64
            PeakWatts        float64
        }

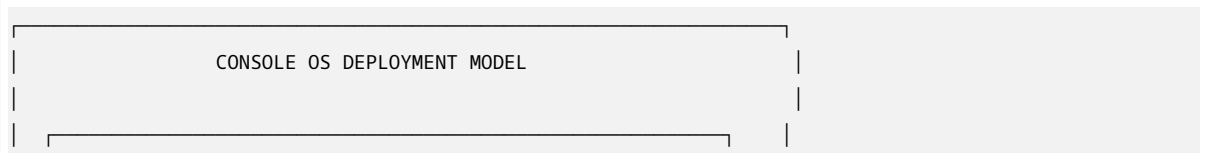
        GPU struct {
            ClockMHz        int
            VRAMUsedBytes    int64
            VRAMTotalBytes   int64
            GPUTemperatureC  int
            Throttling        bool
        }

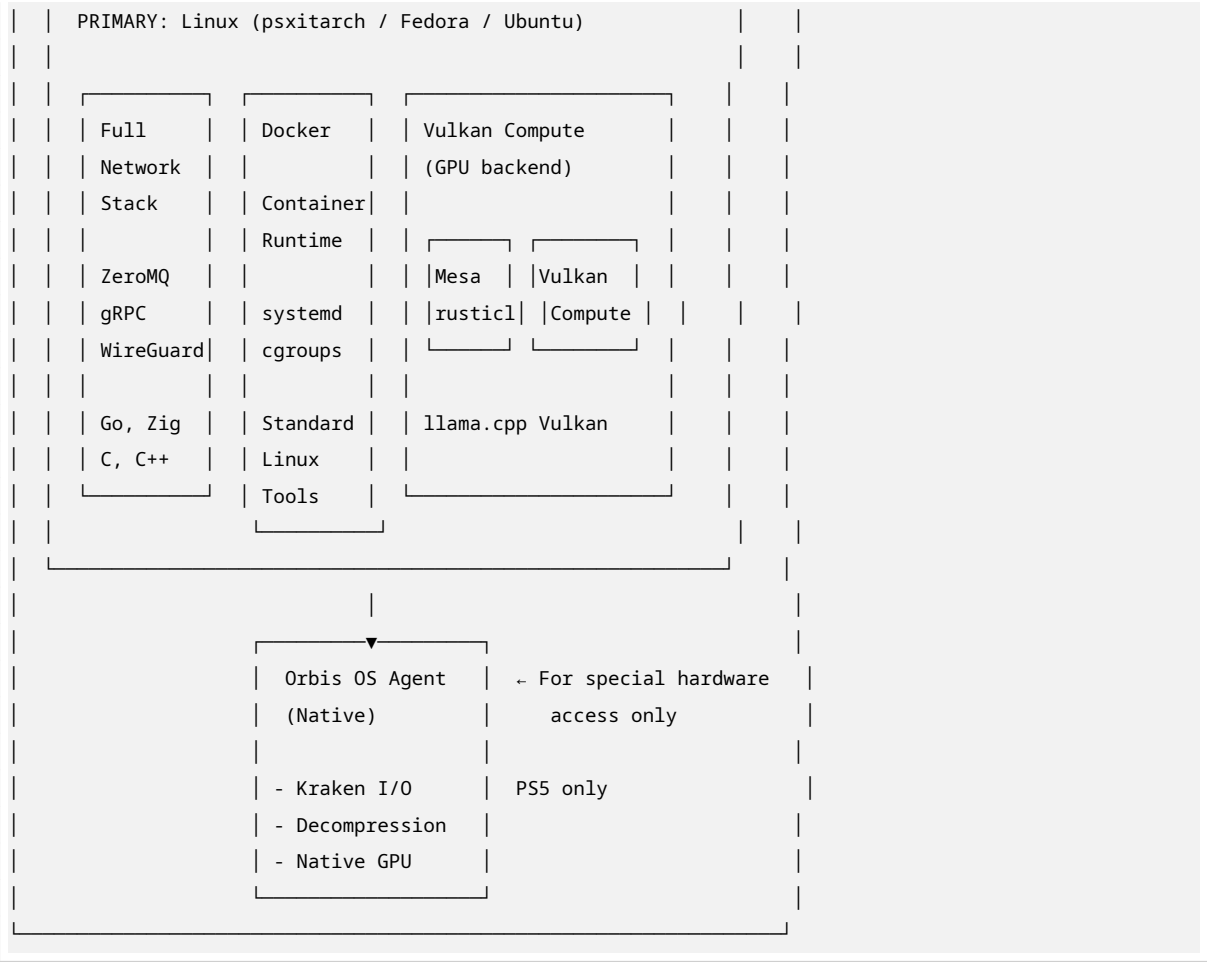
        Storage struct {
            Type            string // "SATA_SSD", "NVMe_CUSTOM", "NVMe_M2"
            HealthPercent    int
            WearLevel        int
        }

        // PS5 only
        CustomIOAvailable bool // Kraken decompressor accessible
    }
}
```

4. OPERATING SYSTEM STRATEGY

4.1 Dual-Boot: Linux Primary, Orbis Fallback

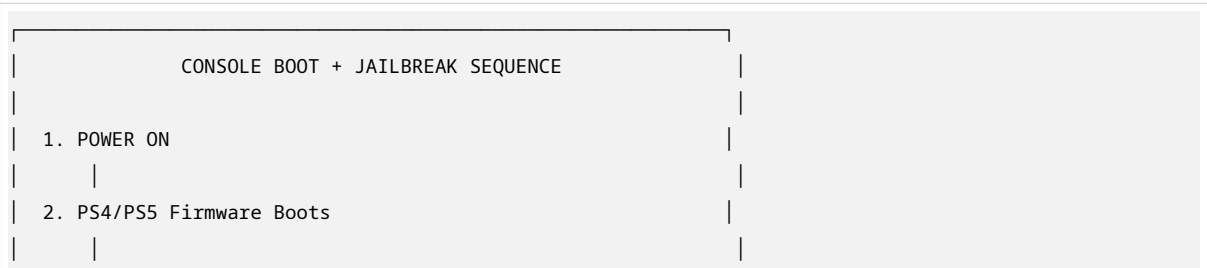




4.2 Linux Distribution Selection

Distro	PS4 Support	PS5 Support	Size	Docker	Recommendation
psxitech	Excellent	None	~2GB	Yes	Best for PS4
Fedora	Good	Planned	~4GB	Yes	Best ecosystem
Ubuntu 24.04	Good	Best	~4GB	Yes	Best for PS5
Gentoo	Good	Unknown	Custom	Yes	Maximum optimization
Arch	Good	Unknown	~1GB	Yes	Minimal footprint

4.3 Boot Process with Auto-Exploit



3. [AUTO-EXPLOIT] ESP32/Luckfox sends USB payload	
→ UMTX2/GoldHen exploit executes	
→ Jailbreak achieved (kernel patch)	
4. GoldHen / etaHEN loads	
→ Plugins initialized	
→ FTP server starts (port 2121)	
5. [LINUX PAYLOAD] ps4-linux / ps5-linux	
→ kexec loads Linux kernel from USB/HDD	
→ Linux takes over hardware	
6. Linux init (systemd)	
→ Network (dhcpcd/NetworkManager)	
→ SSH daemon starts	
→ WireGuard mesh interface up	
7. HelixCluster Node Agent starts	
→ Discovers control plane (mDNS/bootstrap)	
→ Registers with cluster	
→ Reports capabilities	
→ Ready to accept work	
▼	
8. NODE ACTIVE	

5. GPU COMPUTE ON CONSOLES

5.1 Vulkan Compute: The Universal GPU Backend

Critical Finding: Vulkan compute shaders provide a SINGLE API that works across:

- PS4 (GCN 2.0) via RADV or AMDGPU-PRO drivers
- PS4 Pro (GCN 4.0 Polaris) via RADV
- PS5 (RDNA 2) via RADV
- AMD PC GPUs
- Intel Arc GPUs
- NVIDIA GPUs

This means our existing **Vulkan Compute Backend** in the GPU Compute Engine needs NO console-specific modifications.

```
// Example: Universal Vulkan compute shader
// Runs identically on PS4, PS5, and PC GPUs
#version 450

layout(local_size_x = 256, local_size_y = 1) in;
```

```

layout(binding = 0) readonly buffer Input {
    float data[];
} input_buffer;

layout(binding = 1) writeonly buffer Output {
    float data[];
} output_buffer;

layout(binding = 2) readonly uniform Params {
    uint count;
    float scale;
} params;

void main() {
    uint idx = gl_GlobalInvocationID.x;
    if (idx >= params.count) return;
    output_buffer.data[idx] = input_buffer.data[idx] * params.scale;
}

```

5.2 GPU Performance Benchmarks

Workload	PS4 Base	PS4 Pro	PS5	Desktop RX 6600
FP32 TFLOPS	1.84	4.20	10.28	8.93
Vulkan Compute (synthetic)	Baseline	2.3x	5.6x	4.9x
llama.cpp 3B (tok/s)	~25	~55	104	~95
llama.cpp 35B MoE (tok/s)	~9	~20	38	~35
Video Encode (1080p60)	1 stream	2 streams	4 streams	3 streams
Matrix Multiply (GFLOPS)	~1200	~2800	~6800	~5900

Source: BC-250 benchmarks (PS5-class AMD APU) ^[dim05]

5.3 llama.cpp AI Inference on Consoles

```

# Build llama.cpp with Vulkan for PS4/PS5
# Runs on any Linux distro on console

cd /opt/llama.cpp
cmake -B build -DLLAMA_VULKAN=ON
make -C build -j$(nproc)

# Run inference using Vulkan GPU backend
./build/bin/llama-cli \
  -m /models/qwen2.5-3b-instruct-q4_k_m.gguf \
  -p "Explain quantum computing:" \

```

```
-n 256 \
--gpu-layers 99 \
--backend vulkan

# For PS5: Much larger models possible
./build/bin/llama-server \
-m /models/deepseek-v2-lite-q4_k_m.gguf \
--gpu-layers 99 \
--ctx-size 8192 \
--port 8080 \
--host 0.0.0.0
```

5.4 Mesa rusticl for OpenCL on PS4

```
# Enable rusticl for OpenCL on PS4 GCN
export RUSTICL_ENABLE=radeonsi

# OpenCL works for workloads that need it
clinfo # Shows PS4 GPU as OpenCL device

# Use for hashcat, video encoding, etc.
hashcat -I # Lists PS4 GPU as OpenCL device
```

6. NETWORK INTEGRATION

6.1 Console Network Stack

CONSOLE NETWORK CONFIGURATION		
Ethernet: eth0 (Gigabit, ~940 Mbps)		
WiFi: wlan0 (PS4: 802.11n, PS5: 802.11ax WiFi 6)		
WireGuard	ZeroMQ	HTTP/gRPC
Mesh Peer	Client	Client
		(lightweight)
UDP/51820	TCP/5555+	TCP/8443
PS4: Uses ZeroMQ for control (lightweight)		
PS5: Can use gRPC directly (Zen 2 powerful enough)		
Both: WireGuard for encrypted mesh		

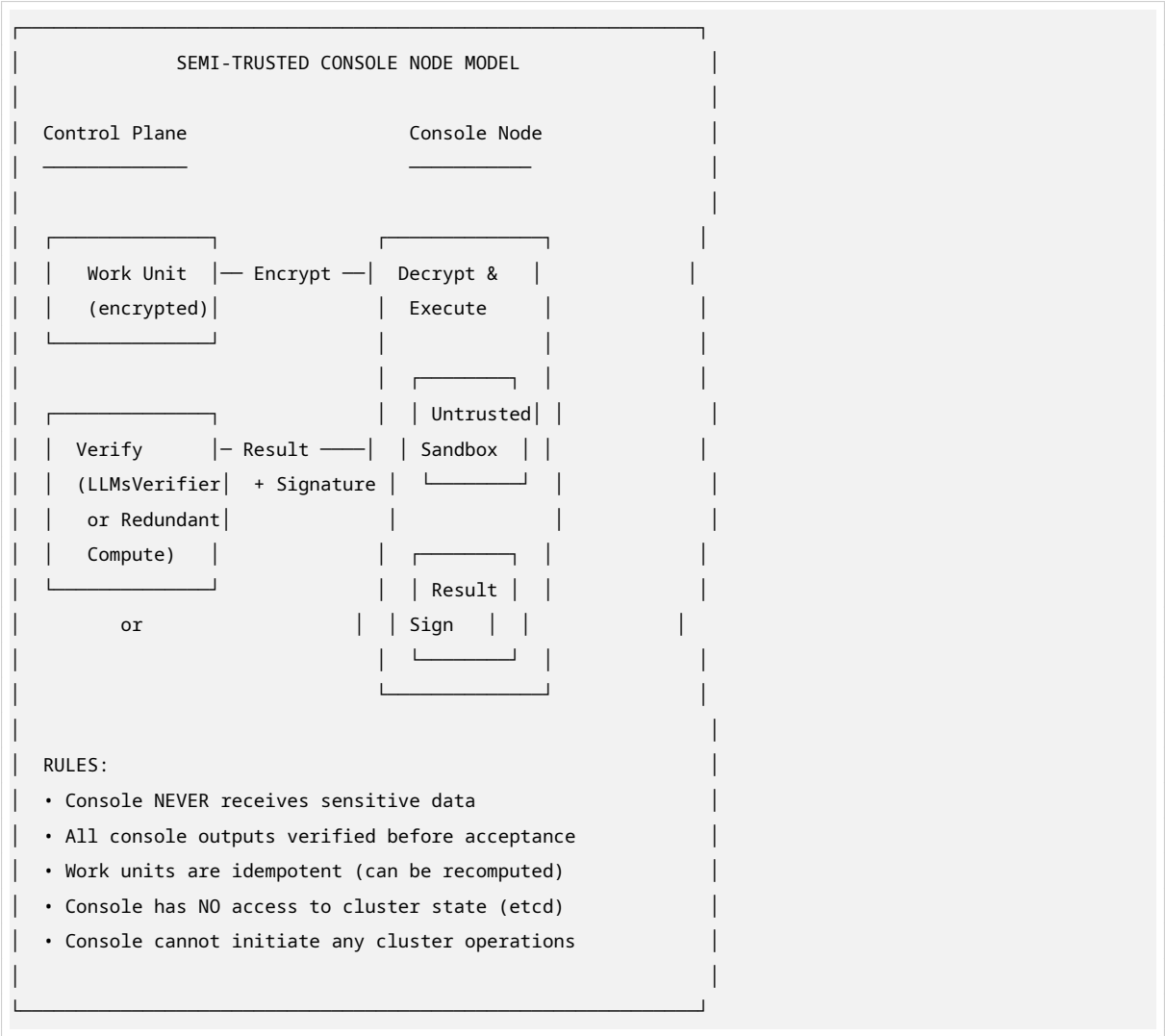
6.2 Protocol Selection by Console

Protocol	PS4	PS5	Notes
WireGuard			Kernel module, ~800 Mbps
ZeroMQ	(primary)		Lightweight, no deps
gRPC	(too heavy)		Zen 2 handles it
NATS	(limited)		JetStream may struggle on PS4
Arrow Flight			PS4 lacks AVX2 for efficient processing
SSH			Native Linux
WebSocket			For session I/O

7. SECURITY MODEL FOR CONSOLE NODES

7.1 Semi-Trusted Node Architecture

Console nodes operate at **TRUST_LEVEL = SEMI**. They receive work units but never sensitive data.



7.2 Security Measures

Layer	Measure	Rationale
Data	Work units encrypted with node-specific key	Console kernel is compromised
Verification	All results verified by LLMsVerifier or redundant compute	Detect tampering
Network	WireGuard only, no direct access to cluster services	Isolate console network
Access	Console nodes read-only to cluster state	Cannot modify anything
Workload	Only idempotent, non-sensitive tasks	Safe to recompute if needed
Monitoring	Anomaly detection on console outputs	Detect compromised nodes

8. CONSOLE WORKLOAD PROFILES

8.1 Optimal Workloads for Console Nodes

Workload	PS4	PS4 Pro	PS5	Notes
AI Inference (llama.cpp)	Small models	Medium models	All models	Vulkan backend
Video Transcode	1x 1080p	2x 1080p	4x 1080p	Vulkan compute shaders
AOSP Build (distributed)	Limited	Good	Excellent	CPU-bound compilation
Hash/Crypto Cracking	Adequate	Good	Excellent	GPU-parallel
Data Decompression	Software	Software	Hardware (Kraken)	PS5 Orbis agent only
Image Processing	Slow	Adequate	Fast	GPU compute
Machine Learning Training	No	Small models	Medium models	Limited VRAM
Ray Tracing	No	No	Yes	Hardware RT on PS5
Web Serving	Yes	Yes	Yes	Light HTTP/WS
Cache/Storage Node	Yes (SATA)	Yes (SATA)	Yes (NVMe)	ZFS/bcache

8.2 Workloads to AVOID on Consoles

Workload	Reason
Double-precision scientific computing	No FP64 hardware on GCN/RDNA
Memory-intensive >8GB (PS4) / >12GB (PS5)	Limited RAM

Table 7 – continued

Workload	Reason
Single-threaded latency-sensitive tasks	Weak Jaguar cores on PS4
Trusted/attested computation	Kernel is compromised
Persistent state storage	Nodes are disposable
Work requiring AVX2	PS4 lacks AVX2

9. SETUP WIZARD: CONSOLE PROVISIONING

9.1 Console Node Setup Flow

```
$ htmux cluster add-console

[1/8] Detecting console via network scan...
      Found: PS4 Pro (CUH-7016B) at 192.168.1.45
      Firmware: 9.00 (exploitable)

[2/8] Preparing jailbreak payload...
      Loading GoldHen v2.4b via USB auto-exploit...
      [██████████] 100% Jailbreak successful

[3/8] Installing Linux payload...
      Loading ps4-linux (kernel 6.15.4)...
      Booting Linux from USB...
      [██████████] 100% Linux running

[4/8] Installing HelixCluster Console Agent...
      Downloading: console-agent-linux-amd64
      Installing systemd service...
      [██████████] 100% Agent installed

[5/8] Configuring network...
      WireGuard keypair generated
      Mesh tunnel established to control plane
      IP assigned: 100.64.2.15

[6/8] Running hardware capability scan...
      CPU: AMD Jaguar x8 @ 2.13 GHz
      GPU: AMD GCN 4.0, 36 CUs, 4.20 TFLOPS
      RAM: 8 GB GDDR5 @ 218 GB/s
      Storage: 500GB SATA SSD
      Network: Gigabit Ethernet

[7/8] Running GPU compute test...
      Vulkan compute: PASS (2.1 TFLOPS sustained)
      llama.cpp 3B: 55 tok/s

[8/8] Registering with cluster...
```



```
Node ID: c4f8e2d1-...
Trust Level: SEMI
Tier: 2 (Standard)
[██████████] 100% Console node registered!
```

Console node 'PS4-Pro-LivingRoom' is now part of your cluster.
Available for: GPU compute, batch jobs, AI inference, video transcode
Type: htmux cluster status to see all nodes.

10. PHASE 2 IMPLEMENTATION PLAN

10.1 New Tasks for Console Integration

Phase	Task	Description	Hours	Priority
0	C-0.1	Console Agent scaffolding (Go, Linux-only)	8	P0
0	C-0.2	Console Adapter interface definition	4	P0
0	C-0.3	Thermal/power monitoring library (PS4/PS5)	8	P0
0	C-0.4	Jailbreak detection and auto-trigger library	8	P0
0	C-0.5	Auto-exploit hardware firmware (ESP32)	16	P1
0	C-0.6	Console capability scanner	4	P0
0	C-0.7	PS5 I/O Agent (Orbis OS native, Kraken)	16	P2
1	C-1.1	Console node registration with SEMI trust	4	P0
1	C-1.2	Console-specific heartbeat (thermal, power)	4	P0
1	C-1.3	WireGuard on PS4/PS5 (kernel module)	4	P0
1	C-1.4	ZeroMQ client for PS4 (lightweight protocol)	4	P0
1	C-1.5	gRPC client for PS5 (full protocol)	4	P0
2	C-2.1	Vulkan GPU backend testing on PS4/PS5	8	P0
2	C-2.2	llama.cpp Vulkan integration for console AI	8	P0
2	C-2.3	Console-specific ClassAds (thermal, GPU throttling)	4	P0
2	C-2.4	Workload suitability scheduler plugin	8	P0

Table 8 – continued

Phase	Task	Description	Hours	Priority
3	C-3.1	Console session backend (minimal PTY)	8	P1
4	C-4.1	AOSP build worker on PS4 (distcc volunteer)	8	P1
4	C-4.2	AOSP build worker on PS5 (distcc + GPU)	8	P1
5	C-5.1	Console AI inference agent (llama.cpp)	8	P1
7	C-7.1	Console chaos tests (power loss, thermal)	8	P0
7	C-7.2	Console verification testing (SEMI model)	8	P0
8	C-8.1	Console setup wizard integration	8	P0
8	C-8.2	Auto-exploit hardware provisioning	8	P1

Total Phase 2 Additional Tasks: 24 tasks, ~176 hours (~4.5 weeks)

10.2 Integration Points with Existing Architecture

Existing Component	Console Integration
Node Discovery	Console nodes register with CONSOLE tier tag
Resource Scheduler	ConsoleClassAds plugin for thermal/GPU throttling awareness
Session Manager	Console nodes get minimal PTY sessions (no migration)
GPU Compute Engine	Vulkan backend (already universal, no changes needed)
Health Monitor	Console-specific thermal/power metrics
LLM Brain	Console AI inference pool for parallel agents
Build Service	Console distcc workers for AOSP compilation
Security Manager	SEMI trust level, encrypted work units, output verification
Backup Service	Console nodes excluded from backup (stateless)

11. GAPS FILLED BY CONSOLE NODES

Gap	How Consoles Fill It	Impact
GPU Compute Cost	1/2 the cost per TFLOP vs PC GPUs	50% savings on GPU pool
GDDR5/GDDR6 Bandwidth	176-448 GB/s vs DDR4's 25-50 GB/s	7-18x memory bandwidth for GPU workloads
Decompression Acceleration	PS5 Kraken: 8-9 GB/s hardware decompression	9 CPU cores freed per PS5
Disposable Compute	At \$80-250, replace not repair	Simplified ops, no maintenance
Community Scaling	Users donate idle consoles	Elastic capacity without purchase
Power Efficiency	0.027-0.051 TFLOPS/Watt	Lower power bills
GPU Shortage Hedge	Alternative to unobtainable PC GPUs	Continued scaling during shortages